

Chapter 11: Duality

Algorithms for Optimization, 2nd ed. (2026)

Kochenderfer & Wheeler

MIT Press

Outline

- 1 11.1 Dual Problem
- 2 11.2 Primal-Dual Methods
- 3 11.3 Dual Ascent
- 4 11.4 ADMM
- 5 11.5 ADMM Applications
- 6 11.6 Distributed Methods
- 7 Summary & Exercises

Motivation: Why Duality?

- Constrained minimisation can be transformed into a **dual maximisation** problem.
- The dual value is always a **lower bound** on the primal optimal value.
- In some cases (strong duality), both problems share the same optimal value – enabling *verification*.
- Dual structure underpins many practical algorithms: interior-point methods, ADMM, distributed optimisation.

Lagrangian Duality Framework

Primal: $\min_{\mathbf{x}} f(\mathbf{x})$ s.t. $g_i(\mathbf{x}) \leq 0, h_j(\mathbf{x}) = 0$

Lagrangian: $\mathcal{L}(\mathbf{x}, \mu, \lambda) = f(\mathbf{x}) + \sum_i \mu_i g_i(\mathbf{x}) + \sum_j \lambda_j h_j(\mathbf{x})$

Dual function: $D(\mu, \lambda) = \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \mu, \lambda)$

Dual problem: $\max_{\mu \geq 0, \lambda} D(\mu, \lambda)$

Weak duality: $d^* \leq p^*$ Strong duality: $d^* = p^*$ (convex + CQ)

The Generalised Lagrangian

General constrained problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{s.t.} \quad \mathbf{g}(\mathbf{x}) \leq \mathbf{0}, \quad \mathbf{h}(\mathbf{x}) = \mathbf{0} \quad (11.1)$$

Generalised Lagrangian (Eq. 11.2)

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \sum_i \mu_i g_i(\mathbf{x}) + \sum_j \lambda_j h_j(\mathbf{x})$$

$\mathbf{x} \in \mathbb{R}^n$ design (primal) variables
 $\boldsymbol{\mu} \geq \mathbf{0}$ inequality multipliers (dual variables)
 $\boldsymbol{\lambda} \in \mathbb{R}^p$ equality multipliers (dual variables)

$\boldsymbol{\mu}$ and $\boldsymbol{\lambda}$ are called **Lagrange multipliers** or **dual variables**.

Primal and Dual Problems

Primal problem (Eq. 11.3)

$$p^* = \min_{\mathbf{x}} \max_{\mu \geq 0, \lambda} \mathcal{L}(\mathbf{x}, \mu, \lambda)$$

The outer *maximisation* sets penalty terms $\rightarrow +\infty$ for infeasible $\mathbf{x}$.

Dual function (Eq. 11.4)

$$\mathcal{D}(\mu, \lambda) = \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \mu, \lambda)$$

Always **concave** in (μ, λ) (lower envelope of affine functions).

Dual problem (Eq. 11.5)

$$d^* = \max_{\mu \geq 0, \lambda} \mathcal{D}(\mu, \lambda)$$

Weak and Strong Duality

Weak Duality (always holds)

$$d^* \leq p^*$$

The dual value is always a *lower bound* on the primal value. The gap $p^* - d^* \geq 0$ is the **duality gap**.

Strong Duality

$$d^* = p^*$$

Holds for **convex** problems satisfying a constraint qualification (e.g. Slater's condition: there exists a strictly feasible point).

Weak vs. Strong Duality

duality gap
↔

Strong duality:
 $d^* = p^*$

d^* (dual) p^* (primal)

Example 11.1: Dual Function as Lower Bound

Problem:

$$\min_x \sin(x) \quad \text{s.t. } x^2 \leq 1$$

Lagrangian: $\mathcal{L}(x, \mu) = \sin(x) + \mu(x^2 - 1)$

Primal:

$$\min_x \max_{\mu \geq 0} [\sin(x) + \mu(x^2 - 1)]$$

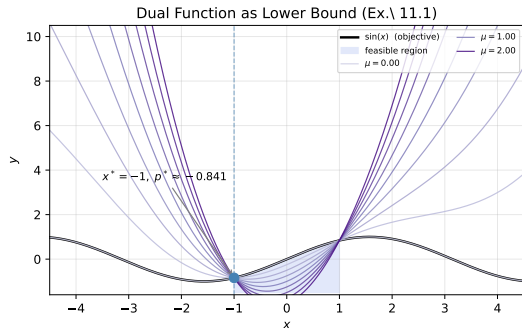
Dual:

$$\max_{\mu \geq 0} \min_x [\sin(x) + \mu(x^2 - 1)]$$

Optimal: $x^* = -1$, $p^* \approx -0.841$.

Each purple curve ($\mathcal{L}$ at fixed μ) has its minimum *below* p^* – weak duality.

Strong duality holds here by inspection.



Example 11.2: Lagrangian Duality with Equality Constraint I

Problem:

$$\min_{\mathbf{x}} x_1 + x_2 + x_1 x_2 \quad \text{s.t. } x_1^2 + x_2^2 = 1$$

Lagrangian: $\mathcal{L}(x_1, x_2, \lambda) = x_1 + x_2 + x_1 x_2 + \lambda(x_1^2 + x_2^2 - 1)$

First-order conditions:

$$\frac{\partial \mathcal{L}}{\partial x_1} = 1 + x_2 + 2\lambda x_1 = 0, \quad \frac{\partial \mathcal{L}}{\partial x_2} = 1 + x_1 + 2\lambda x_2 = 0, \quad x_1^2 + x_2^2 - 1 = 0$$

Four critical points:

x_1	x_2	λ	$f(\mathbf{x})$
-1	0	1/2	-1
0	-1	1/2	-1
$\frac{\sqrt{2}+1}{\sqrt{2}+2}$	$\frac{\sqrt{2}+1}{\sqrt{2}+2}$	$\frac{1}{2}(-1 - \sqrt{2})$	≈ 1.914
$\frac{\sqrt{2}-1}{\sqrt{2}-2}$	$\frac{\sqrt{2}-1}{\sqrt{2}-2}$	$\frac{1}{2}(-1 + \sqrt{2})$	≈ -0.914

Example 11.2: Lagrangian Duality with Equality Constraint II

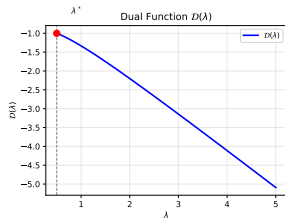
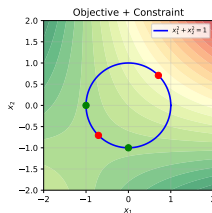
Optimal solutions: $[-1, 0]$ and $[0, -1]$ (value -1).

Using substitution $u = x_1 + x_2$, $v = x_1 - x_2$:

$$\mathcal{D}(\lambda) = \min_{u,v} \frac{1}{4}(2\lambda + 1)u^2 + u + \frac{1}{4}(2\lambda - 1)v^2 - \lambda$$

$$\mathcal{D}(\lambda) = \begin{cases} -\left(\frac{1}{2\lambda + 1} + \lambda\right) & \lambda \geq \frac{1}{2} \\ -\infty & \text{otherwise} \end{cases}$$

Strong duality holds: $d^* = -1 = p^*$.



Primal-Dual Interior-Point: Motivation

- **Primal-dual methods** simultaneously update primal $\mathbf{x}$ and dual μ .
- Build on interior-point barrier method for:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{s.t.} \quad \mathbf{g}(\mathbf{x}) \leq \mathbf{0} \quad (11.7)$$

- Replace with log-barrier problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) - \frac{1}{\rho} \sum_i \log(-g_i(\mathbf{x})) \quad (11.9)$$

- Setting $\mu_i^* = -1/(\rho g_i(\mathbf{x}^*))$ recovers Lagrangian KKT:

$$\mathbf{0} = \nabla f(\mathbf{x}^*) + \sum_i \mu_i^* \nabla g_i(\mathbf{x}^*) \quad (11.13)$$

- **Duality gap** at iteration: $p^* - \mathcal{D}(\mu^*) = m/\rho \rightarrow 0$ as $\rho \rightarrow \infty$.

Primal-Dual System and Newton Step

Seek to satisfy *stationarity* and *complementarity* simultaneously:

$$\mathbf{r}(\mathbf{x}, \boldsymbol{\mu}, \rho) = \begin{bmatrix} \nabla f(\mathbf{x}) + \sum_i \mu_i \nabla g_i(\mathbf{x}) \\ -\mu_1 g_1(\mathbf{x}) - 1/\rho \\ \vdots \\ -\mu_m g_m(\mathbf{x}) - 1/\rho \end{bmatrix} = \mathbf{0} \quad (11.19)$$

Newton (descent) direction

$$\mathbf{d} = -\mathbf{J}^{-1} \mathbf{r}$$

where $\mathbf{J} = \partial \mathbf{r} / \partial (\mathbf{x}, \boldsymbol{\mu})$.

Step-size safeguards:

- Bound α to keep $\boldsymbol{\mu} > 0$: $\alpha \leq (1 - \varepsilon)\mu_i / (-d_{\mu_i})$ if $d_{\mu_i} < 0$.
- Reduce α while any $g_i(\mathbf{x} + \alpha \mathbf{d}_x) > 0$ (maintain feasibility).
- Armijo sufficient decrease on $\|\mathbf{r}\|$.

Example 11.3: Primal-Dual Descent Direction

Problem: $\min_x x^2$ s.t. $(x - 3)^2 \leq 1$ (feasible: $x \in [2, 4]$)

Residual vector:

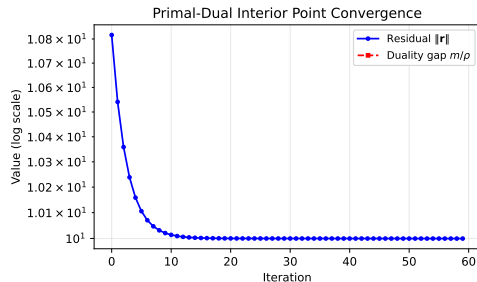
$$\mathbf{r} = \begin{bmatrix} 2x + 2\mu(x - 3) \\ -\mu(x - 3)^2 + \mu - 1/\rho \end{bmatrix} = \mathbf{0}$$

Jacobian:

$$\mathbf{J} = \begin{bmatrix} 2 + 2\mu & 2(x - 3) \\ -2\mu(x - 3) & 1 - (x - 3)^2 \end{bmatrix}$$

Descent direction: $\mathbf{d} = -\mathbf{J}^{-1}\mathbf{r}$, where d_1 updates x and d_2 updates μ .

The primal-dual method converges quickly because it simultaneously enforces both stationarity and the complementary slackness condition.



Python: Primal-Dual Interior-Point (Algorithm 11.2)

```
1 import numpy as np
2 from scipy.linalg import solve
3
4 def primal_dual_interior_point(f, gradf, hessf, gs, gradgs, hessgs,
5                               rho, x, mu, alpha_max=1.0, beta=1e-4,
6                               gamma=2.0, eps=0.01, p=0.9,
7                               res_min=1e-4, gap_min=1e-4):
8     """
9     Primal-dual interior point method for:
10     minimize f(x)    subject to g_i(x) <= 0    for i in 1..m
11     """
12     m = len(gs)
13     def residual(x, mu, rho):
14         r_x = gradf(x) + sum(mu[i]*gradgs[i](x) for i in range(m))
15         r_mu = [-mu[i]*gs[i](x) - 1/rho for i in range(m)]
16         return np.concatenate([r_x, r_mu])
17
18     res = np.linalg.norm(residual(x, mu, rho))
19     gap = sum(-mu[i]*gs[i](x) for i in range(m))
20
21     while res > res_min or gap > gap_min:
22         rho *= gamma
23         # Build Jacobian and solve Newton system
24         n = len(x)
25         J = np.zeros((n+m, n+m))
26         J[:n, :n] = hessf(x) + sum(mu[i]*hessgs[i](x) for i in range(m))
27         for i in range(m):
```

Dual Ascent: Alternating Optimisation

When strong duality holds, an optimum $(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*)$ satisfies:

$$\mathcal{L}(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*) = \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*) \quad \text{and} \quad \mathcal{L}(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*) = \max_{\boldsymbol{\mu} \geq \mathbf{0}, \boldsymbol{\lambda}} \mathcal{L}(\mathbf{x}^*, \boldsymbol{\mu}, \boldsymbol{\lambda})$$

Dual Ascent Algorithm

- 1 **Minimise** Lagrangian over $\mathbf{x}$ (primal update): $\mathbf{x}^{(k+1)} = \arg \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}^{(k)}, \boldsymbol{\lambda}^{(k)})$
- 2 **Gradient ascent** on dual variables (dual update):

$$\boldsymbol{\mu}^{(k+1)} = \max(\mathbf{0}, \boldsymbol{\mu}^{(k)} + \alpha \mathbf{g}(\mathbf{x}^{(k+1)}))$$

$$\boldsymbol{\lambda}^{(k+1)} = \boldsymbol{\lambda}^{(k)} + \alpha \mathbf{h}(\mathbf{x}^{(k+1)})$$

Gradient of $\mathcal{D}$ w.r.t. $\boldsymbol{\mu}, \boldsymbol{\lambda}$ is the constraint violation.

- Dual function is always *concave* $\Rightarrow$ gradient ascent converges.
- Slow convergence for ill-conditioned problems (zig-zagging).

Dual Ascent: Numerical Example

Problem: $\min x_1^2 + x_2^2$ s.t. $x_1 + x_2 = 1$

Dual function:

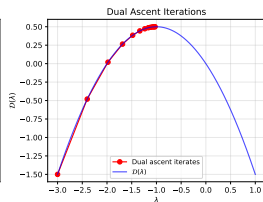
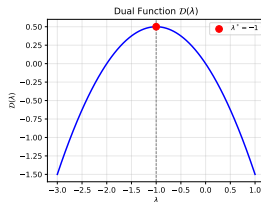
$$\mathcal{D}(\lambda) = -\frac{\lambda^2}{2} - \lambda$$

Maximised at $\lambda^* = -1$ giving $\mathcal{D}(-1) = \frac{1}{2} = p^*$.

Update rule:

$$x_i^{(k+1)} = -\lambda^{(k)}/2$$

$$\lambda^{(k+1)} = \lambda^{(k)} + \underbrace{\alpha (x_1 + x_2 - 1)}_{\text{constraint viol.}}$$



Method of Multipliers (Augmented Lagrangian)

Dual ascent can converge slowly. Adding a quadratic penalty helps:

Augmented Lagrangian

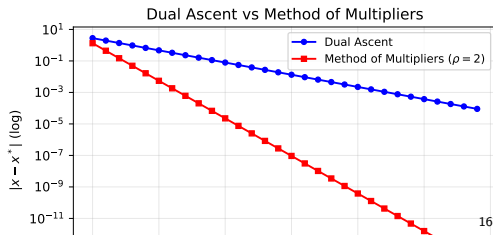
$$\mathcal{L}_\rho(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\lambda}^\top \mathbf{h}(\mathbf{x}) + \frac{\rho}{2} \|\mathbf{h}(\mathbf{x})\|_2^2$$

Method of Multipliers Updates

$$\mathbf{x}^{(k+1)} = \arg \min_{\mathbf{x}} \mathcal{L}_\rho(\mathbf{x}, \boldsymbol{\lambda}^{(k)})$$

$$\boldsymbol{\lambda}^{(k+1)} = \boldsymbol{\lambda}^{(k)} + \rho \mathbf{h}(\mathbf{x}^{(k+1)})$$

- Converges for a fixed $\rho > 0$ (unlike plain penalty method).
- Quadratic term enforces feasibility more aggressively.
- Drawback: cannot easily parallelise the joint



ADMM: Alternating Direction Method of Multipliers

ADMM Canonical Form

$$\min_{\mathbf{x}_1, \mathbf{x}_2} f_1(\mathbf{x}_1) + f_2(\mathbf{x}_2) \quad \text{s.t.} \quad \mathbf{A}_1 \mathbf{x}_1 + \mathbf{A}_2 \mathbf{x}_2 = \mathbf{b} \quad (11.30)$$

Augmented Lagrangian:

$$\mathcal{L}_\rho(\mathbf{x}_1, \mathbf{x}_2, \boldsymbol{\lambda}) = f_1(\mathbf{x}_1) + f_2(\mathbf{x}_2) + \boldsymbol{\lambda}^\top (\mathbf{A}_1 \mathbf{x}_1 + \mathbf{A}_2 \mathbf{x}_2 - \mathbf{b}) + \frac{\rho}{2} \|\mathbf{A}_1 \mathbf{x}_1 + \mathbf{A}_2 \mathbf{x}_2 - \mathbf{b}\|_2^2$$

ADMM Updates (Eqs. 11.35–11.37)

$$\mathbf{x}_1^{(k+1)} = \arg \min_{\mathbf{x}_1} \mathcal{L}_\rho(\mathbf{x}_1, \mathbf{x}_2^{(k)}, \boldsymbol{\lambda}^{(k)})$$

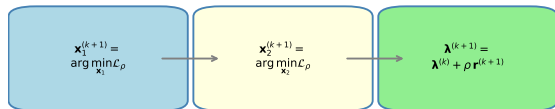
$$\mathbf{x}_2^{(k+1)} = \arg \min_{\mathbf{x}_2} \mathcal{L}_\rho(\mathbf{x}_1^{(k+1)}, \mathbf{x}_2, \boldsymbol{\lambda}^{(k)})$$

$$\boldsymbol{\lambda}^{(k+1)} = \boldsymbol{\lambda}^{(k)} + \rho (\mathbf{A}_1 \mathbf{x}_1^{(k+1)} + \mathbf{A}_2 \mathbf{x}_2^{(k+1)} - \mathbf{b})$$

Key: alternating direction gives ADMM its name and enables efficient parallel computation.

ADMM Block Diagram & Convergence

ADMM Iteration Structure



Stopping criteria:

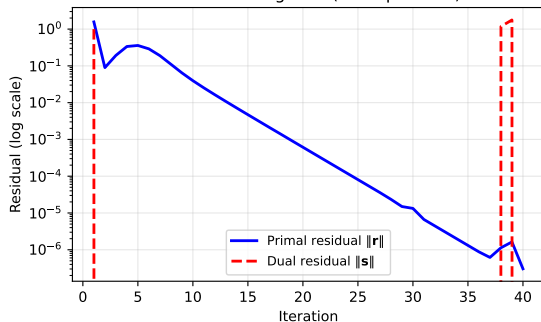
- **Primal residual:**

$$\mathbf{r}^{(k+1)} = \mathbf{A}_1 \mathbf{x}_1^{(k+1)} + \mathbf{A}_2 \mathbf{x}_2^{(k+1)} - \mathbf{b}$$

- **Dual residual:** $\mathbf{s}^{(k+1)} = \rho \mathbf{A}_2^\top (\mathbf{x}_2^{(k+1)} - \mathbf{x}_2^{(k)})$

Terminate when $\|\mathbf{r}\| < \varepsilon_r$ and $\|\mathbf{s}\| < \varepsilon_s$.

ADMM Convergence (Example 11.4)



- ρ is increased when $\|\mathbf{r}\| \gg \|\mathbf{s}\|$
- Decreases ρ when $\|\mathbf{s}\| \gg \|\mathbf{r}\|$
- Self-adaptive ρ improves convergence.

Scaled ADMM Form

Introduce the **scaled dual variable** $\mathbf{u} = (1/\rho)\boldsymbol{\lambda}$:

$$\mathcal{L}_\rho(\mathbf{x}_1, \mathbf{x}_2, \boldsymbol{\lambda}) = f_1(\mathbf{x}_1) + f_2(\mathbf{x}_2) + \frac{\rho}{2} \|\mathbf{A}_1\mathbf{x}_1 + \mathbf{A}_2\mathbf{x}_2 - \mathbf{b} + \mathbf{u}\|_2^2 - \frac{\rho}{2} \|\mathbf{u}\|_2^2$$

Scaled ADMM Updates (Eqs. 11.50–11.52)

$$\mathbf{x}_1^{(k+1)} = \arg \min_{\mathbf{x}_1} f_1(\mathbf{x}_1) + \frac{\rho}{2} \|\mathbf{A}_1\mathbf{x}_1 - \mathbf{v}^{(k)}\|_2^2, \quad \mathbf{v}^{(k)} = -\mathbf{A}_2\mathbf{x}_2^{(k)} + \mathbf{b} - \mathbf{u}^{(k)}$$

$$\mathbf{x}_2^{(k+1)} = \arg \min_{\mathbf{x}_2} f_2(\mathbf{x}_2) + \frac{\rho}{2} \|\mathbf{A}_2\mathbf{x}_2 - \mathbf{w}^{(k)}\|_2^2, \quad \mathbf{w}^{(k)} = -\mathbf{A}_1\mathbf{x}_1^{(k+1)} + \mathbf{b} - \mathbf{u}^{(k)}$$

$$\mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} + \mathbf{A}_1\mathbf{x}_1^{(k+1)} + \mathbf{A}_2\mathbf{x}_2^{(k+1)} - \mathbf{b} = \mathbf{u}^{(0)} + \sum_{j=1}^{k+1} \mathbf{r}^{(j)}$$

The scaled dual variable is the **running sum of primal residuals**.

Python: ADMM Core Implementation

```
1 import numpy as np
2
3 def admm(f1_prox, f2_prox, A1, A2, b, rho=1.0, gamma=1.5,
4         eps_r=1e-3, eps_s=1e-3, max_iter=200):
5     """
6     ADMM solver for:  $\min f_1(x_1) + f_2(x_2) \quad \text{s.t.} \quad A_1 x_1 + A_2 x_2 = b$ 
7
8     f1_prox(v, rho): proximal operator of f1 --  $\operatorname{argmin} f_1(x) + \rho/2 \|x - v\|^2$ 
9     f2_prox(v, rho): proximal operator of f2
10    """
11    n1 = A1.shape[1]; n2 = A2.shape[1]
12    x1 = np.zeros(n1); x2 = np.zeros(n2); u = np.zeros(len(b))
13
14    for k in range(max_iter):
15        # x1-update: minimise  $f_1(x_1) + \rho/2 \|A_1 x_1 - v\|^2$ 
16        v = -A2 @ x2 + b - u
17        x1 = f1_prox(A1, v, rho)      # application-specific solver
18
19        # x2-update: minimise  $f_2(x_2) + \rho/2 \|A_2 x_2 - w\|^2$ 
20        w = -A1 @ x1 + b - u
21        x2 = f2_prox(A2, w, rho)
22
23        # dual (scaled) variable update
24        r = A1 @ x1 + A2 @ x2 - b      # primal residual
25        s = rho * A1.T @ (x2 - x1)     # dual residual (simplified)
26        u = u + r
27
```

11.5.1 General Constrained Optimisation in ADMM Form

Any constrained problem can be cast in ADMM form:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{s.t. } \mathbf{x} \in \mathcal{X} \quad \implies \quad \min_{\mathbf{x}_1, \mathbf{x}_2} \underbrace{f(\mathbf{x}_1)}_{f_1} + \underbrace{\mathcal{K}_{\mathcal{X}}(\mathbf{x}_2)}_{f_2} \quad \text{s.t. } \mathbf{x}_1 - \mathbf{x}_2 = \mathbf{0}$$

where $\mathcal{K}_{\mathcal{X}}(\mathbf{x}_2) = 0$ if $\mathbf{x}_2 \in \mathcal{X}$, else $+\infty$.

Proximal Minimisation

$$\arg \min_{\mathbf{x}} f(\mathbf{x}) + \frac{\rho}{2} \|\mathbf{x} - \mathbf{p}\|_2^2$$

If f is differentiable, linearising gives a gradient step:

$$\mathbf{x} \approx \mathbf{p} - \frac{1}{\rho} \nabla f(\mathbf{p})$$

The $\mathbf{x}_2$ -update becomes: **project** $\mathbf{x}_1^{(k+1)} + \mathbf{u}^{(k)}$ onto $\mathcal{X}$.

11.5.2 Alternating Projections

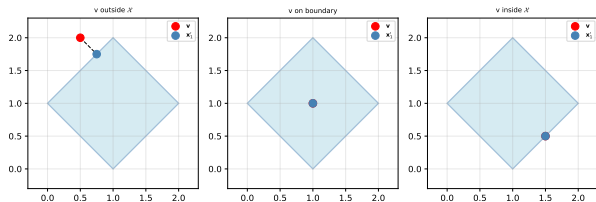
Goal: find $\mathbf{x} \in \mathcal{X}_1 \cap \mathcal{X}_2$

ADMM form:

$$\min_{\mathbf{x}_1, \mathbf{x}_2} \|\mathbf{x}_1\|_{\mathcal{X}_1} + \|\mathbf{x}_2\|_{\mathcal{X}_2} \quad \text{s.t. } \mathbf{x}_1 - \mathbf{x}_2 = \mathbf{0}$$

ADMM updates: each is a **projection** onto the respective convex set.

Proximal Minimization: Projection onto $\mathcal{X}$



- **Von Neumann:** alternate projections directly, no dual variable. Slow in narrow canyons.
- **Dykstra (= ADMM):** dual variable $\mathbf{u}$ acts as *momentum*. Faster convergence.

11.5.3 Least Absolute Deviations (LAD)

Problem: $\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_1$

Cast in ADMM form: $\min_{\mathbf{x}_1, \mathbf{x}_2} f_1(\mathbf{x}_1) + \|\mathbf{x}_2\|_1 \quad \text{s.t.} \quad \mathbf{Ax}_1 - \mathbf{x}_2 = \mathbf{b}$

Soft Thresholding Operator S_κ

$$(S_\kappa(\mathbf{x}))_i = \begin{cases} x_i + \kappa & x_i < -\kappa \\ 0 & |x_i| \leq \kappa \\ x_i - \kappa & x_i > \kappa \end{cases} = \text{sign}(x_i) \max(|x_i| - \kappa, 0)$$

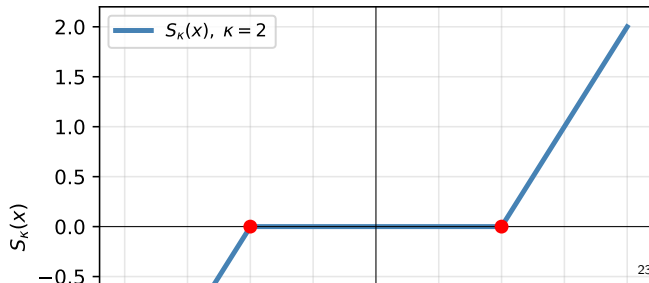
ADMM updates:

$$\mathbf{x}'_1 = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top (\mathbf{b} + \mathbf{x}_2 - \mathbf{u})$$

$$\mathbf{x}'_2 = S_{1/\rho}(\mathbf{Ax}'_1 - \mathbf{b} + \mathbf{u})$$

$$\mathbf{u}' = \mathbf{u} + \mathbf{Ax}'_1 - \mathbf{b} - \mathbf{x}'_2$$

Soft Thresholding Operator ($\kappa = 2$)



11.5.4 Basis Pursuit (Split Bregman)

Problem: $\min_{\mathbf{x}} \|\mathbf{x}\|_1$ s.t. $\mathbf{Ax} = \mathbf{b}$ (find sparse solutions)

ADMM updates (Eqs. 11.79–11.82):

$$\mathbf{x}'_1 = (\mathbf{I} - \mathbf{A}^\top (\mathbf{AA}^\top)^{-1} \mathbf{A})(\mathbf{x}_2 - \mathbf{u}) + \mathbf{A}^\top (\mathbf{AA}^\top)^{-1} \mathbf{b}$$

$$\mathbf{x}'_2 = S_{1/\rho}(\mathbf{x}'_1 + \mathbf{u}) \quad \mathbf{u}' = \mathbf{u} + \mathbf{x}'_1 - \mathbf{x}'_2$$

- Precompute $\mathbf{I} - \mathbf{A}^\top (\mathbf{AA}^\top)^{-1} \mathbf{A}$ and $\mathbf{A}^\top (\mathbf{AA}^\top)^{-1} \mathbf{b}$ once.
- Each iteration is cheap: one matrix-vector product + soft thresholding.
- This is the **split Bregman** method.

11.5.5 Lasso: L_1 -Regularised Regression

Lasso problem:

$$\min_{\mathbf{x}} \frac{1}{2} \|\mathbf{Ax} - \mathbf{b}\|_2^2 + \lambda \|\mathbf{x}\|_1 \quad (11.83)$$

$\lambda > 0$ biases toward **sparse** solutions.

ADMM form: $f_1(\mathbf{x}_1) = \frac{1}{2} \|\mathbf{Ax}_1 - \mathbf{b}\|_2^2$, $f_2(\mathbf{x}_2) = \lambda \|\mathbf{x}_2\|_1$,
s.t. $\mathbf{x}_1 - \mathbf{x}_2 = \mathbf{0}$.

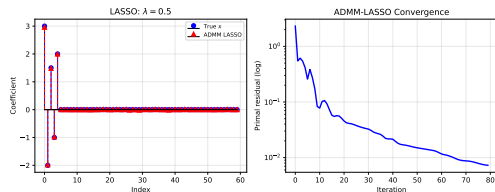
Updates (Eqs. 11.85–11.87):

$$\mathbf{x}'_1 = (\mathbf{A}^\top \mathbf{A} + \rho \mathbf{I})^{-1} (\mathbf{A}^\top \mathbf{b} + \rho(\mathbf{x}_2 - \mathbf{u}))$$

$$\mathbf{x}'_2 = S_{\lambda/\rho}(\mathbf{x}'_1 + \mathbf{u})$$

$$\mathbf{u}' = \mathbf{u} + \mathbf{x}'_1 - \mathbf{x}'_2$$

Matrix $\mathbf{A}^\top \mathbf{A} + \rho \mathbf{I}$ always invertible for $\rho > 0$.



Python: ADMM-Lasso Implementation (Algorithm 11.7)

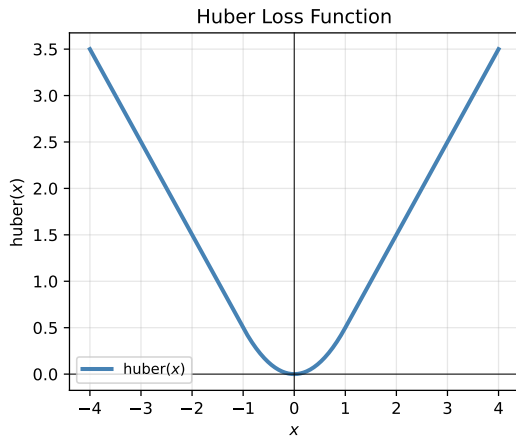
```
1 import numpy as np
2
3 def lasso_admm(A, b, lam, x1=None, x2=None, rho=1.0,
4               gamma=2.0, eps=1e-3, max_iter=100):
5     """
6     Solve  $\min 0.5 \|Ax - b\|^2 + \text{lam} \|x\|_1$ 
7     via ADMM (Algorithm 11.7 in Python).
8     """
9     n = A.shape[1]
10    if x1 is None: x1 = np.zeros(n)
11    if x2 is None: x2 = np.zeros(n)
12    u = np.zeros(n)
13
14    AtA = A.T @ A
15    Atb = A.T @ b
16
17    def soft(v, kappa):
18        return np.sign(v) * np.maximum(np.abs(v) - kappa, 0)
19
20    for k in range(max_iter):
21        # x1-update: linear system solve (warm-startable)
22        x1 = np.linalg.solve(AtA + rho * np.eye(n),
23                             Atb + rho * (x2 - u))
24        # x2-update: soft thresholding
25        x2 = soft(x1 + u, lam / rho)
26        # scaled dual update
27        u = u + x1 - x2
```

Huber Loss and Other ADMM Applications

Huber loss:

$$\text{huber}(x) = \begin{cases} \frac{1}{2}x^2 & |x| \leq 1 \\ |x| - \frac{1}{2} & \text{otherwise} \end{cases}$$

- Quadratic for small deviations (smooth).
- Linear for large deviations (robust to outliers).
- x_2 -update:
$$\mathbf{x}'_2 = \frac{\rho}{1+\rho}(\mathbf{Ax}'_1 - \mathbf{b} + \mathbf{u}) + \frac{1}{1+\rho}S_{1+1/\rho}(\mathbf{Ax}'_1 - \mathbf{b} + \mathbf{u})$$



ADMM application summary:

Application	f_2
Constrained opt.	indicator $\mathbb{I}_{\mathcal{X}}$
Quadratic	" "

Consensus Optimisation

Goal: minimise sum of k convex objectives:

$$\min_{\mathbf{x}} f_1(\mathbf{x}) + f_2(\mathbf{x}) + \cdots + f_k(\mathbf{x}) \quad (11.88)$$

ADMM Consensus Reformulation

$$\min_{\mathbf{x}_1^{(1:k)}, \mathbf{x}_2} \sum_{i=1}^k f_i(\mathbf{x}_1^{(i)}) \quad \text{s.t. } \mathbf{x}_1^{(i)} = \mathbf{x}_2 \quad \forall i$$

All copies of $\mathbf{x}_1^{(i)}$ must agree with the global $\mathbf{x}_2$.

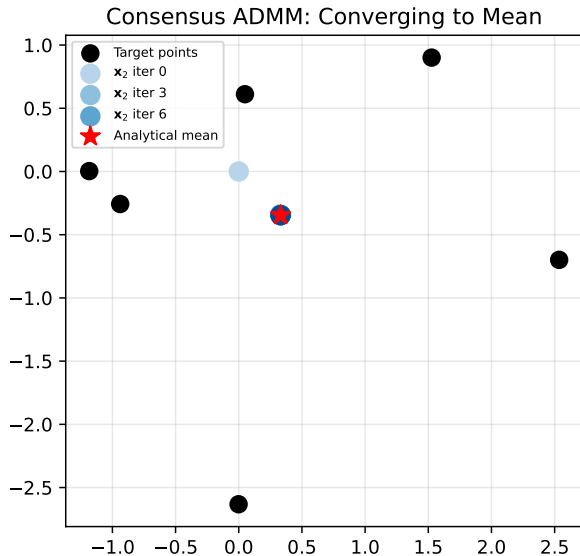
Consensus updates (Eqs. 11.90–11.92):

$$(\mathbf{x}_1^{(i)})' = \arg \min_{\mathbf{x}_1^{(i)}} f_i(\mathbf{x}_1^{(i)}) + (\boldsymbol{\lambda}^{(i)})^\top (\mathbf{x}_1^{(i)} - \mathbf{x}_2) + \frac{\rho}{2} \|\mathbf{x}_1^{(i)} - \mathbf{x}_2\|_2^2 \quad (\text{parallelisable!})$$

$$\mathbf{x}_2' = \frac{1}{k} \sum_{i=1}^k \left[(\mathbf{x}_1^{(i)})' + \frac{1}{\rho} \boldsymbol{\lambda}^{(i)} \right] \quad (\text{consensus / averaging step})$$

$$(\boldsymbol{\lambda}^{(i)})' = \boldsymbol{\lambda}^{(i)} + \rho \left[(\mathbf{x}_1^{(i)})' - \mathbf{x}_2' \right]$$

Consensus ADMM: Convergence to Mean



Key properties:

- Each $\mathbf{x}_1^{(i)}$ -update can run on a **separate processor**.
- Only $\mathbf{x}_1^{(i)}$, $\lambda^{(i)}$, $\mathbf{x}_2$, ρ need to be communicated.
- The implementation details of f_i need not be shared.
- Privacy-preserving applications in federated learning.

As $\rho \rightarrow \infty$: $\mathbf{x}_2 \rightarrow \bar{\mathbf{x}}$ (mean of all designs).

Python: Consensus ADMM (Algorithm 11.8)

```
1 import numpy as np
2 from concurrent.futures import ThreadPoolExecutor
3
4 def consensus_admm(objectives, x1_inits, x2_init,
5                    rho=1.0, gamma=2.0, eps_r=1e-3, eps_s=1e-3,
6                    max_iter=100):
7     """
8     Consensus ADMM: min sum_i f_i(x) (f_i in objectives list)
9     Each f_i is a callable returning (value, gradient).
10    Uses scipy.optimize.minimize for each x1^(i) update.
11    """
12    from scipy.optimize import minimize as sci_min
13    k = len(objectives)
14    x1s = [np.array(xi, dtype=float) for xi in x1_inits]
15    x2 = np.array(x2_init, dtype=float)
16    lams = [np.zeros_like(x2) for _ in range(k)]
17
18    x1_bar_old = np.mean(x1s, axis=0)
19
20    for it in range(max_iter):
21        # Parallel x1^(i) updates
22        def update_i(args):
23            i, f_i, x1, lam = args
24            def obj(x):
25                val = f_i(x)
26                penalty = lam @ (x - x2) + 0.5*rho*np.sum((x - x2)**2)
27                return val + penalty
```

11.6.2 General Consensus Optimisation

When local vectors $\mathbf{x}_1^{(i)}$ need only contain a *subset* of $\mathbf{x}_2$:

$$\min_{\mathbf{x}_1^{(1:k)}, \mathbf{x}_2} \sum_{i=1}^k f_i(\mathbf{x}_1^{(i)}) \quad \text{s.t. } \mathbf{x}_1^{(i)} = \mathbf{x}_2^{(i)} \quad \forall i$$

General Consensus Updates

$$(\mathbf{x}_1^{(i)})' = \arg \min_{\mathbf{x}_1^{(i)}} f_i(\mathbf{x}_1^{(i)}) + (\boldsymbol{\lambda}^{(i)})^\top (\mathbf{x}_1^{(i)} - \mathbf{x}_2^{(i)}) + \frac{\rho}{2} \|\mathbf{x}_1^{(i)} - \mathbf{x}_2^{(i)}\|_2^2$$

$$(\mathbf{x}_2)_g' = \frac{1}{n_g} \sum_{i=1}^k \sum_j [(\mathbf{x}_1^{(i)})'_j \text{ where component } j \text{ maps to } g]$$

$$(\boldsymbol{\lambda}^{(i)})' = \boldsymbol{\lambda}^{(i)} + \rho \left((\mathbf{x}_1^{(i)})' - (\mathbf{x}_2^{(i)})' \right)$$

n_g = number of local vectors whose entry corresponds to global component g .

Example 11.7: Bakery Placement as General Consensus

Problem: Place 3 bakeries ($\mathbf{b}^{(1)}, \mathbf{b}^{(2)}, \mathbf{b}^{(3)}$) to serve 6 establishments.

- 2 cafes at $\mathbf{e}^{(1)} = [0, 0]$, $\mathbf{e}^{(2)} = [0.5, 0]$
- 2 brunch spots at $\mathbf{e}^{(3)} = [3, 2]$, $\mathbf{e}^{(4)} = [0.5, 3]$
- 2 restaurants at $\mathbf{e}^{(5)} = [3.5, -0.5]$, $\mathbf{e}^{(6)} = [2.5, 0.7]$

Bakery 1 (croissants) serves cafes + brunch; bakery 2 (baguettes) serves brunch + restaurants; bakery 3 (dinner rolls) serves restaurants only.

Global variable: $\mathbf{x}_2 = [b_1^{(1)}, b_2^{(1)}, b_1^{(2)}, b_2^{(2)}, b_1^{(3)}, b_2^{(3)}]^\top$

Each establishment optimises a *local* vector that only contains entries for its compatible bakeries $\Rightarrow$ true distributed computation.

Chapter 11 Summary

- **Dual problem:** always concave; its value $\leq p^*$ (weak duality).
- **Strong duality** ($d^* = p^*$): holds for convex problems + CQ.
- **Primal-dual interior-point:** simultaneously drives residual and duality gap to zero.
- **Dual ascent:** gradient ascent on concave $\mathcal{D}$; convergence guaranteed but slow.
- **Method of multipliers:** augmented Lagrangian gives faster, fixed- ρ convergence.
- **ADMM:** decomposes method of multipliers into alternating updates; each sub-problem cheaper.
- **ADMM applications:** alternating projections, LAD, basis pursuit, lasso, Huber regression.
- **Consensus:** parallelises ADMM over k processors; only averages communicated.
- **General consensus:** each agent sees only relevant components – privacy-preserving.

Method	Primal update	Dual update
Dual ascent	Lagrangian min	Gradient step
Method of multipliers	Aug. Lagrangian min	Multiplier step
ADMM	Two alternating min	Multiplier step
Consensus ADMM	k parallel min	Mean + multiplier

Selected Exercises I

Exercise 11.1. Show complementary slackness $\mu_i^* g_i(\mathbf{x}^*) = 0$ holds when the duality gap is zero.

Key idea: Zero gap $\Rightarrow d^* = p^* = f(\mathbf{x}^*)$. Since $\mathcal{D}(\boldsymbol{\mu}^*, \boldsymbol{\lambda}^*) \leq \mathcal{L}(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*)$ and each term $\mu_i^* g_i(\mathbf{x}^*) \leq 0$, the sum must be exactly zero $\Rightarrow$ complementary slackness.

Exercise 11.2. Is the dual of the dual equal to the primal?

Answer: Not necessarily. From $d^* \leq p^*$ and $dd^* \leq d^*$, we get $dd^* \leq p^*$. If a duality gap exists, $dd^* \neq p^*$.

Selected Exercises II

Exercise 11.3. ADMM $\mathbf{x}_1$ -update for quadratic $f_1(\mathbf{x}_1) = \frac{1}{2}\mathbf{x}_1^\top \mathbf{Q}\mathbf{x}_1 + \mathbf{q}^\top \mathbf{x}_1 + q$ (positive semidefinite $\mathbf{Q}$):

$$(\mathbf{Q} + \rho \mathbf{A}_1^\top \mathbf{A}_1)\mathbf{x}'_1 = \rho \mathbf{A}_1^\top (\mathbf{A}_2 \mathbf{x}_2 - \mathbf{b}) - \mathbf{q} - \lambda$$

Exercise 11.6. Simplify consensus $\mathbf{x}_2$ -update: let $\bar{\mathbf{x}}_1 = (1/k) \sum_i (\mathbf{x}_1^{(i)})'$. Then:

$$\mathbf{x}'_2 = \bar{\mathbf{x}}_1 + \frac{1}{\rho} \bar{\lambda}$$

This removes the need for an explicit $\mathbf{x}_2$ variable.

Algorithm Complexity Summary

Algorithm	Per-iteration cost	Key parameter
Primal-dual IP	$O((n + m)^3)$ Newton solve	$\rho \uparrow$ each iter
Dual ascent	$O(\text{cost of } \min_{\mathbf{x}} \mathcal{L})$	step size α
Method of multipliers	$O(\text{cost of aug. Lag. min})$	$\rho > 0$ fixed
ADMM (basic)	$O(n^3)$ per sub-problem	ρ, γ
ADMM-Lasso	$O(n^3)$ (one-time Chol.) + $O(n^2)$	ρ, λ
Consensus ADMM	$O(k \cdot \text{sub-prob})$ parallel	ρ, γ

Practical guidelines:

- Use adaptive ρ : scale up when $\|\mathbf{r}\| \gg \|\mathbf{s}\|$, down otherwise.
- Initialise with small ρ and increase geometrically ($\rho \leftarrow \gamma\rho$).
- Precompute matrix factorisations when possible (LASSO, basis pursuit).
- ADMM typically shows linear convergence; near optimal solutions may need refinement.

References

- Kochenderfer & Wheeler, *Algorithms for Optimization*, 2nd ed., MIT Press, 2026.
- S. Boyd & L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.
- S. J. Wright, *Primal-Dual Interior-Point Methods*, SIAM, 1997.
- S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein, "Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers," *Foundations and Trends in Machine Learning*, vol. 3, no. 1, 2011.
- R. Tibshirani, "Regression Shrinkage and Selection via the Lasso," *JRSS-B*, vol. 58, 1996.
- T. Goldstein & S. Osher, "The Split Bregman Method for L_1 -Regularized Problems," *SIAM Journal on Imaging Sciences*, vol. 2, 2009.